PenDown

A Personal Journal & Productivity Web Application

Software Engineering (HNDIT 4012)

Software Requirements Specification

Done by:

	Name	Index
1	R. Anithajanu	NAW/IT/2324/F/19
2	P. Kishonika	NAW/IT/2324/F/225
3	G. Roshalini	NAW/IT/2324/F/35
4	M.F.F. Zafra	NAW/IT/2324/F/38
5	Mysara	NAW/IT/2324/F/17

2026

Table of Contents

List of Figure

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for PenDown, a personal journal and productivity web application. The purpose of this document is to provide a clear and detailed description of the system's intended behavior, features, and constraints, so that the design and implementation can be evaluated against a well-defined set of requirements. This document is intended for academic evaluation as part of a software engineering assignment, and serves as a reference for understanding the scope, architecture, and functionality of the system.

1.2 Scope

PenDown is a single-user, browser-based web application that allows a registered user to write and manage private journal entries, create and track personal to-do tasks, pin important journal entries and tasks for quick access, and manage basic profile settings including a username and a light/dark theme preference.

The system is built using front-end web technologies only — HTML, CSS, and JavaScript — and uses the browser's built-in localStorage as its data persistence layer. There is no server-side backend, and no external database. The application is designed to run entirely client-side within a modern web browser.

The scope of the system includes the following core modules:

- User registration and authentication
- A personal dashboard providing an overview of the user's activity
- Journal entry management (create, view, edit, delete, pin)
- To-do task management (create, edit, delete, mark status, pin)
- A consolidated view of all pinned journal entries and tasks
- Profile management, including username editing, theme switching, and account deletion

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
MVC	Model-View-Controller, a software architectural pattern
UI	User Interface
localStorage	A browser-based key-value storage mechanism used to persist data on the client side
Client-side	Logic and processing that occurs within the user's web browser, as opposed to on a remote server
CRUD	Create, Read, Update, delete — the four basic operations performed on stored data

2. Overall Description

2.1 Product Features

- Account registration and login with input validation
- Password visibility toggle on authentication forms
- A dashboard, summarizing journal and task statistics, with a live date/time display
- Full CRUD functionality for journal entries, including automatic word count
- Full CRUD functionality for to-do tasks, including completion status tracking
- Pinning/unpinning of journal entries and tasks, with a dedicated Pinned page
- Editable username, and light/dark theme preference
- Account deletion with confirmation

2.2 User Classes and Characteristics

The system supports a single user class: the Registered User. Every person who creates an account has identical permissions and access to all features of the system — there is no administrator role, and no distinction in privilege between users. Each user's data (journal entries, tasks, and theme preference) is stored separately and is not accessible to other users of the same system.

2.3 Operating Environment

- Any modern desktop or mobile web browser with JavaScript and localStorage support (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Brave)
- No server-side runtime, or database server is required

2.4 Assumptions and Dependencies

- It is assumed that the user's browser has localStorage enabled and has not disabled or restricted it (e.g., via private/incognito settings that block persistent storage).
- It is assumed that only one user is actively logged in per browser at any given time, since session state is stored in a single localStorage key.
- It is assumed that the application files are served consistently from the same origin (ideally via a local development server) to ensure reliable localStorage behaviour.
- The system depends on the Google Fonts service (for the Fraunces and Inter typefaces) being reachable; if unavailable, the browser will fall back to default system fonts without affecting functionality.
- It is assumed the user does not manually clear their browser's site data, as this would permanently erase their account and all associated journal entries and tasks.

3. System Architecture

3.1 Client-Server Architecture

Although PenDown does not use a traditional remote server, it is conceptually structured around a client-server model to separate concerns clearly:

- Client: The web browser, which renders all HTML/CSS pages, executes JavaScript logic, and captures user interaction (form input, button clicks, navigation).
- Server (local data layer): The browser's localStorage, which acts as the persistent data store for user accounts, journal entries, tasks, and theme preferences. All data read/write operations pass through a single dedicated JavaScript file (storage.js), which acts as the sole interface to this data layer — mirroring how a client application would communicate with a server-side data API.

This separation means that if the project were extended in the future, the storage.js module could be replaced with actual HTTP requests to a real backend server with minimal changes to the rest of the application, since all other files already treat it as an abstracted data service rather than accessing localStorage directly.

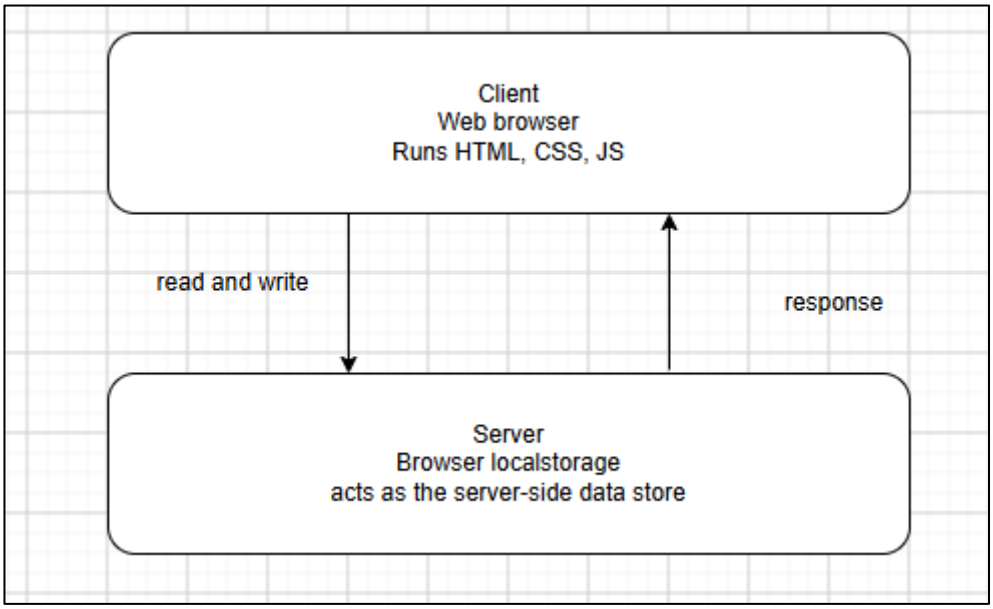

Figure 3.1.1: Client-Server architecture

3.2 MVC (Model-View-Controller) Architecture

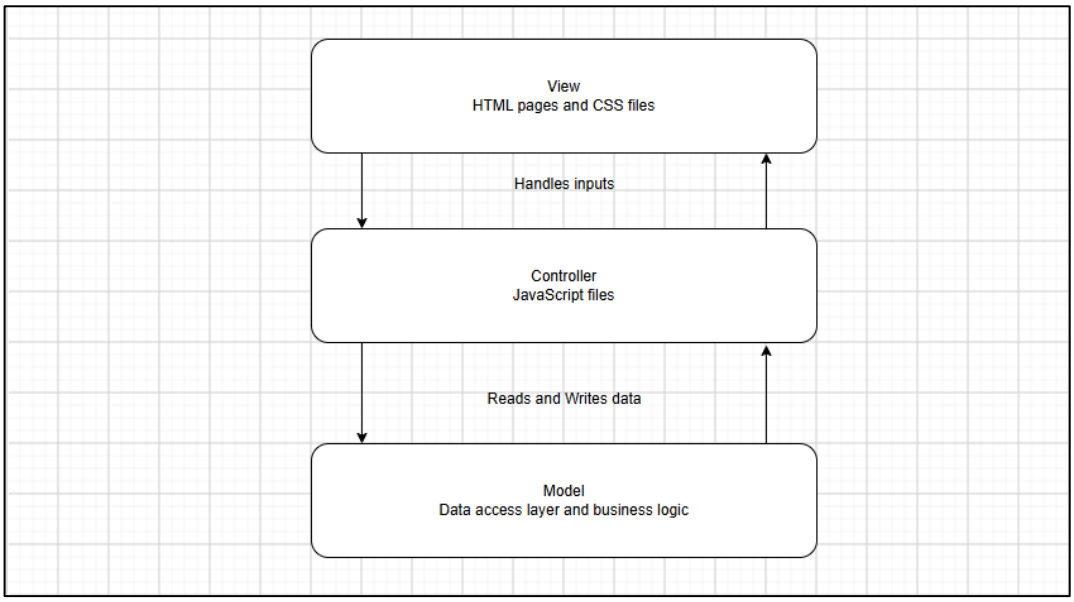

Figure 3.2.1: MVC architecture

4. User Interface Design

4.1 User Interfaces

	Interface	Description
1	Home Page	Public landing page introducing the application, with links to Login and Register
2	Login Page	Authentication form for existing users, with password visibility toggle
3	Register Page	Account creation form with validation for username, email, and password
4	Dashboard Page	Post-login landing page showing a welcome message, live date/time, statistics, and recent activity
5	Journal Page	Interface for creating, viewing, editing, deleting, and pinning journal entries
6	To-Do Page	Interface for creating, editing, deleting, completing, and pinning tasks
7	Pinned Page	Consolidated view of all pinned journal entries and tasks
8	Profile Page	Interface for editing username, toggling theme, logging out, and deleting the account

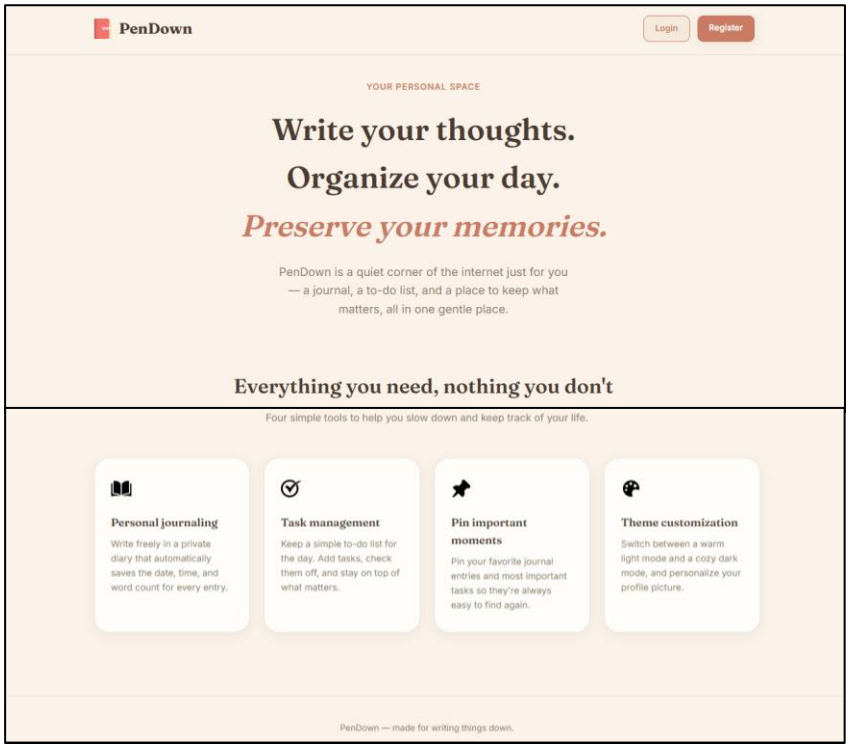

Figure 4.1.1: Home page

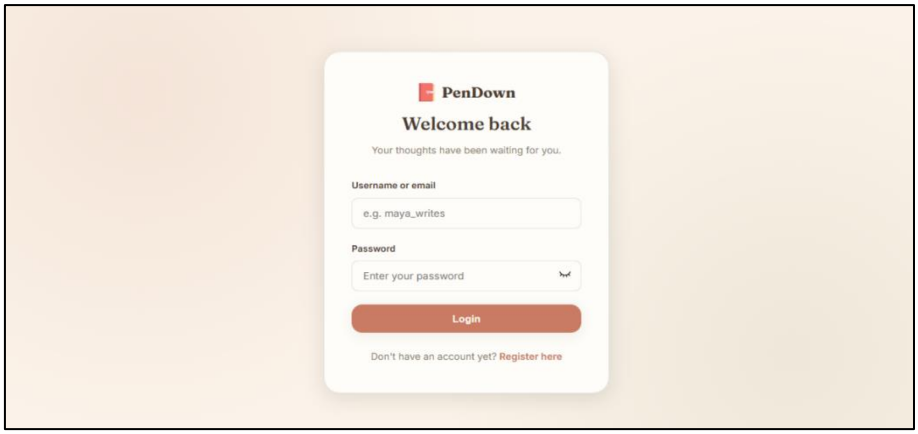

Figure 4.1.2: Login page

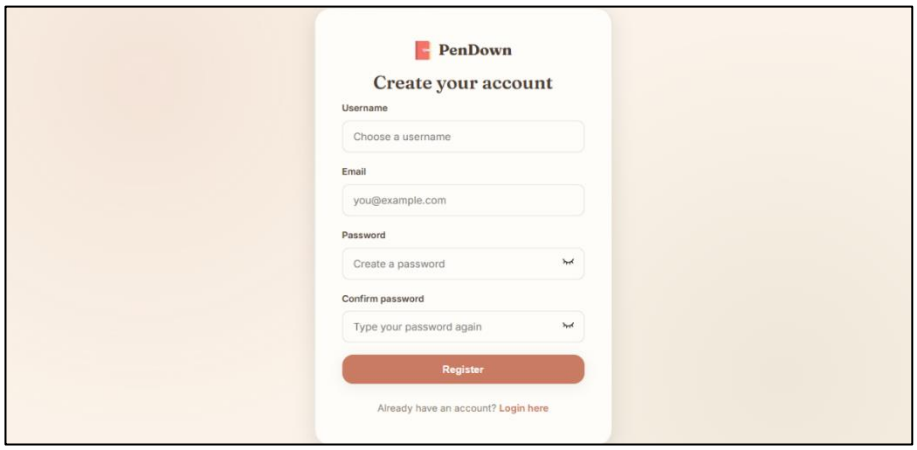

Figure 4.1.3: Register page

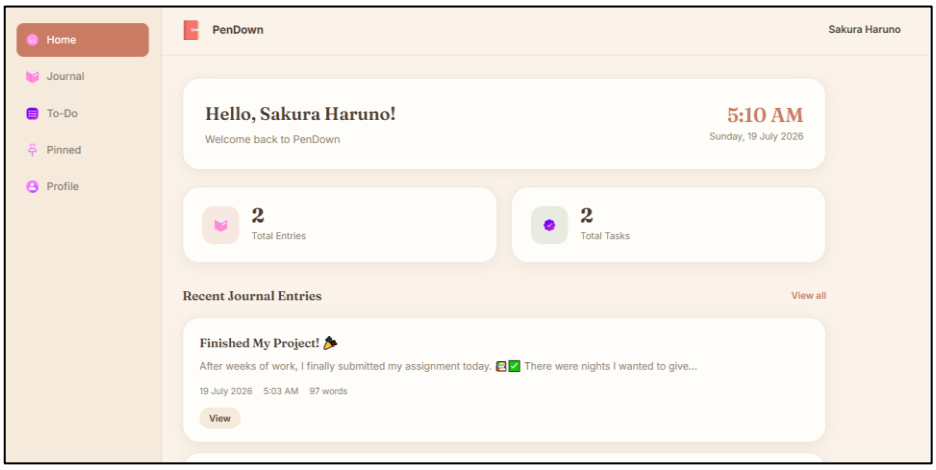

Figure 4.1.4: Dashboard page

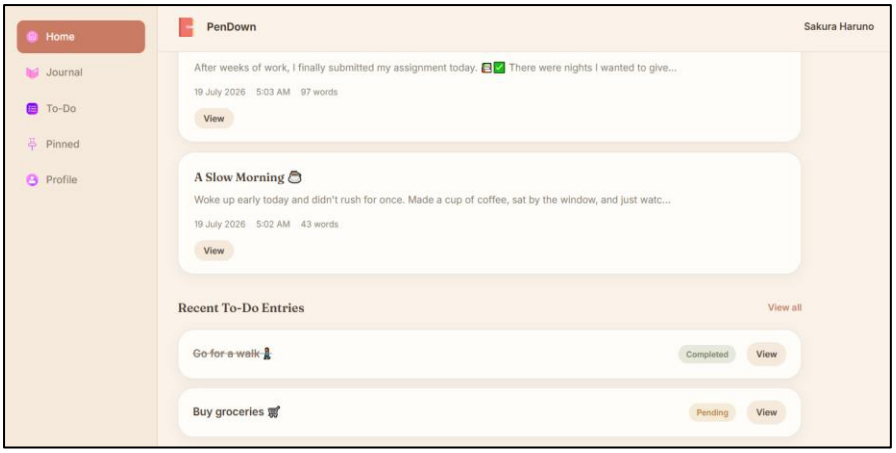

Figure 4.1.5: Home navigation

Figure 4.1.6: Journal navigation

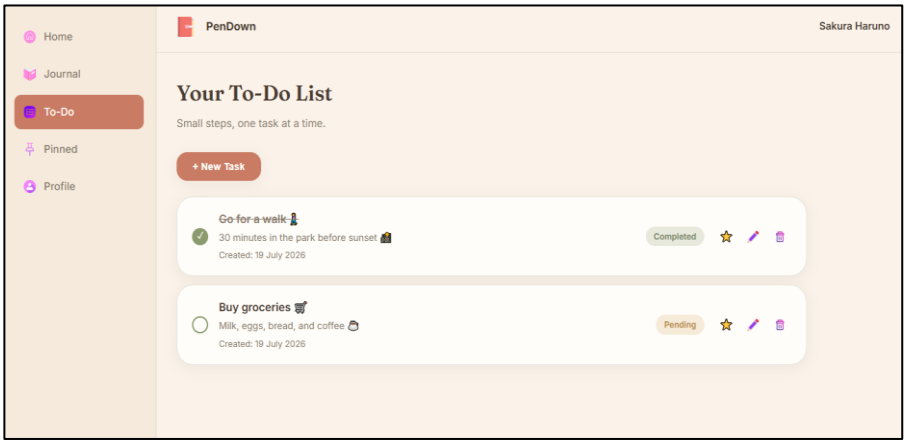

Figure 4.1.7: To-Do navigation

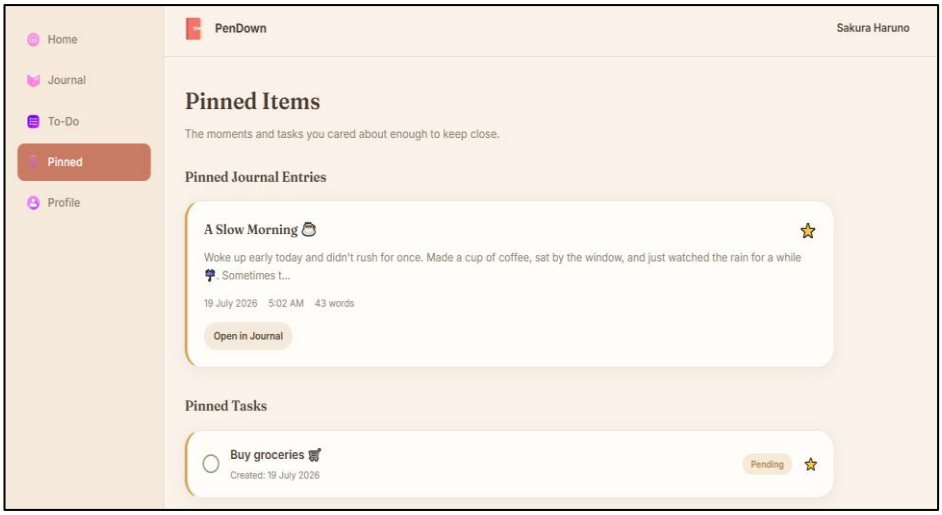

Figure 4.1.8: Pinned navigation

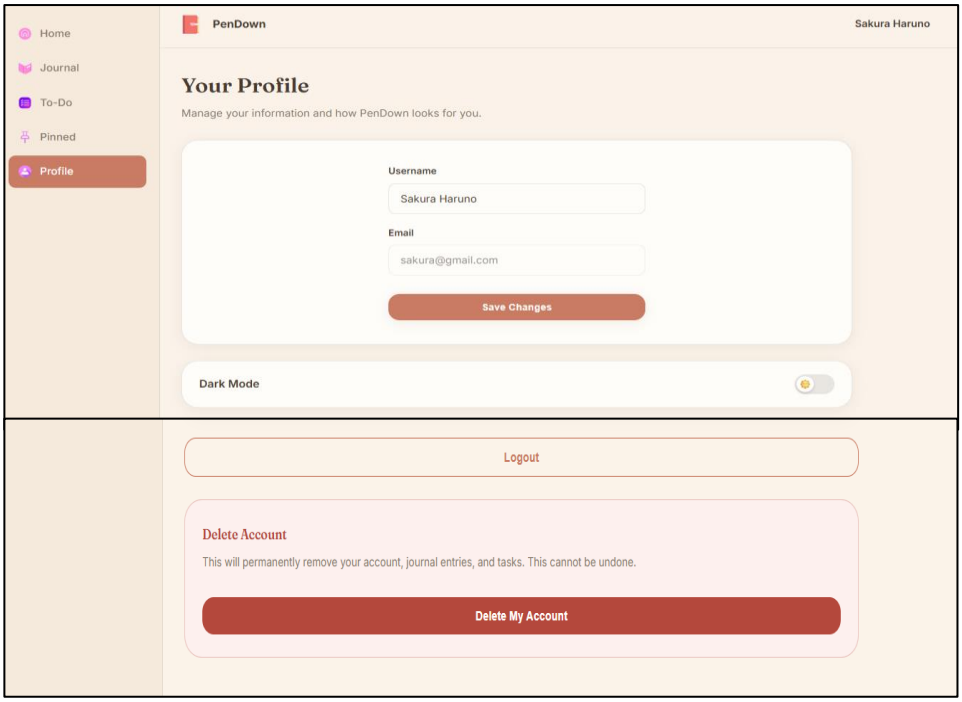

Figure 4.1.9: Profile navigation

4.2 Wireframes

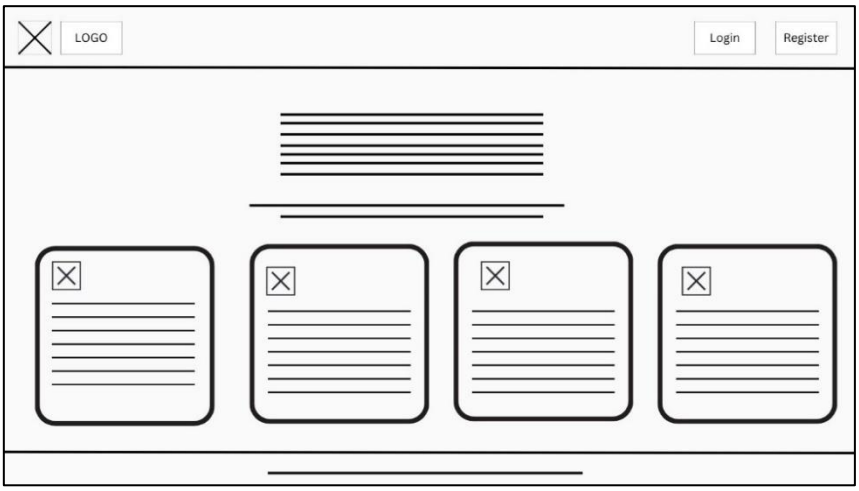

Figure 4.2.1: Home wireframe

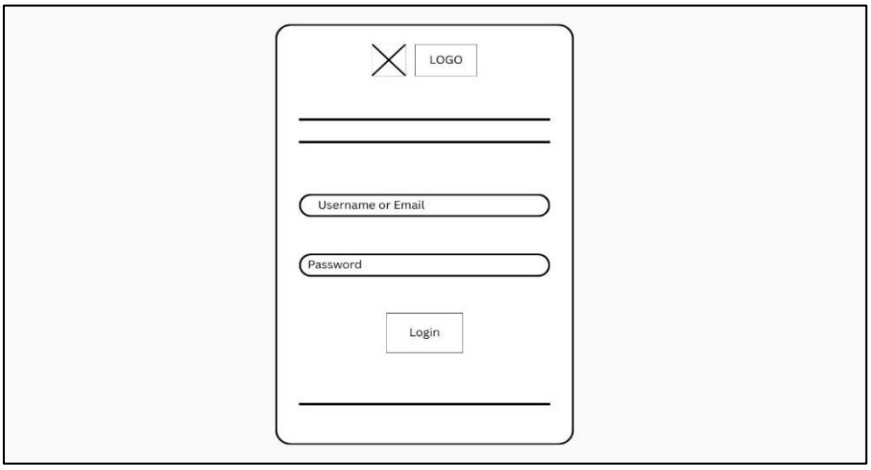

Figure 4.2.2: Login wireframe

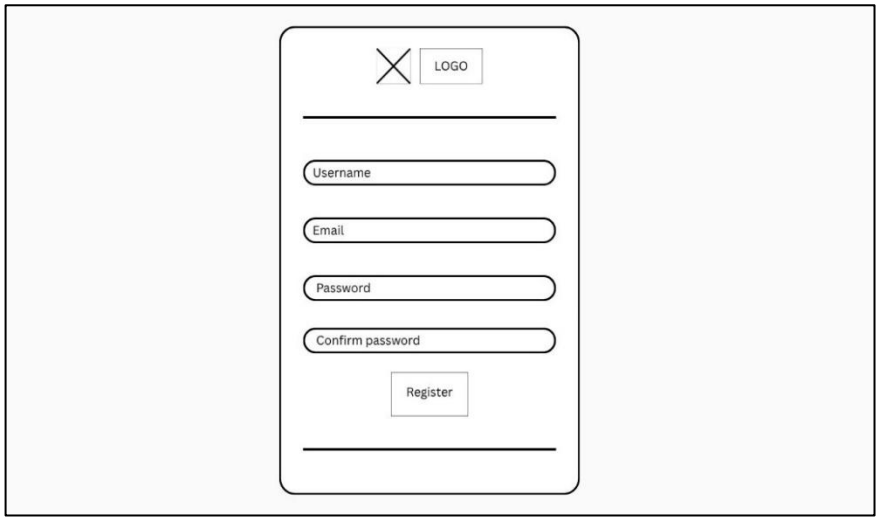

Figure 4.2.3: Register wireframe

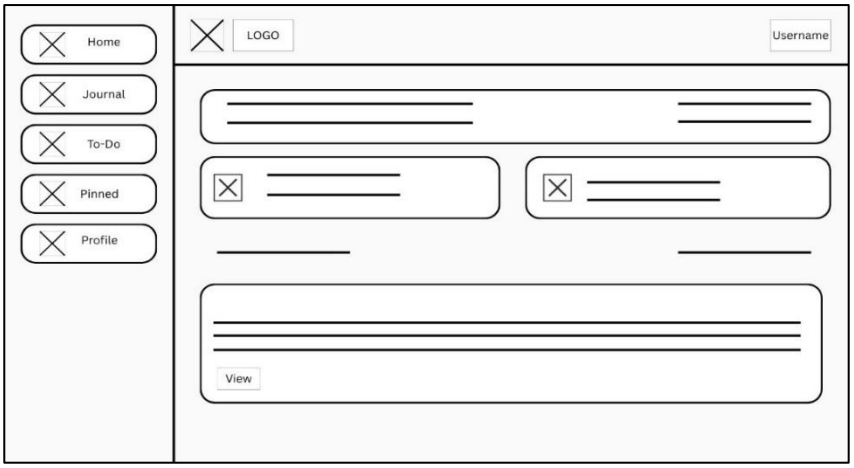

Figure 4.2.4: Dashboard wireframe

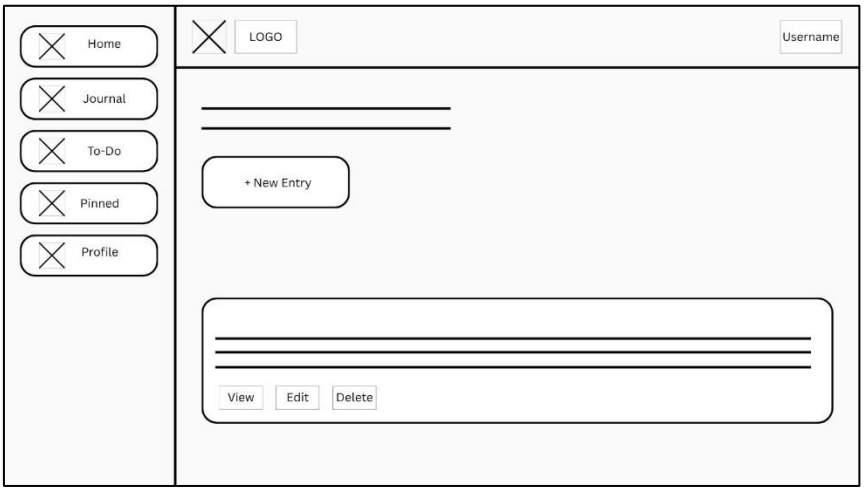

Figure 4.2.5: Journal wireframe

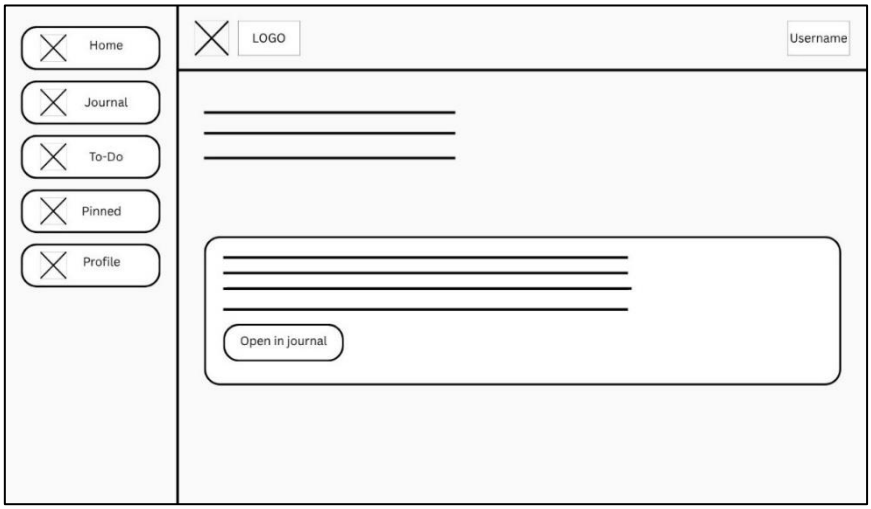

Figure 4.2.6: Pinned wireframe

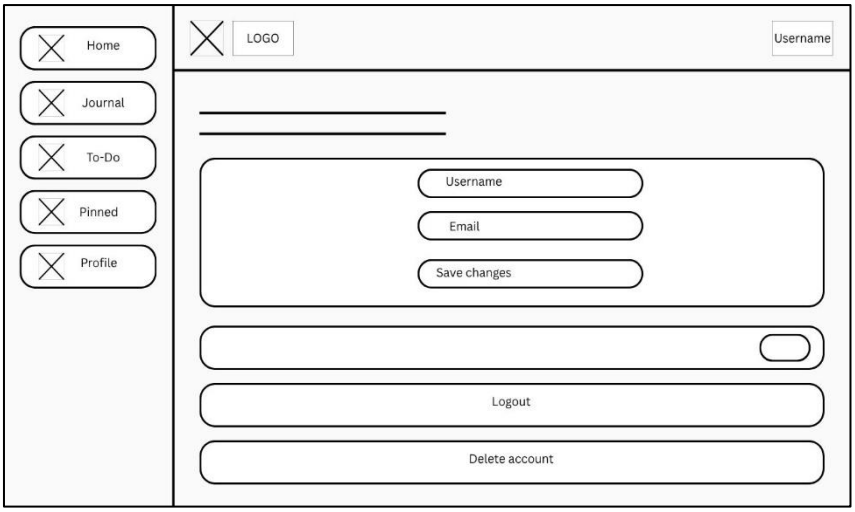

Figure 4.2.7: Profile wireframe

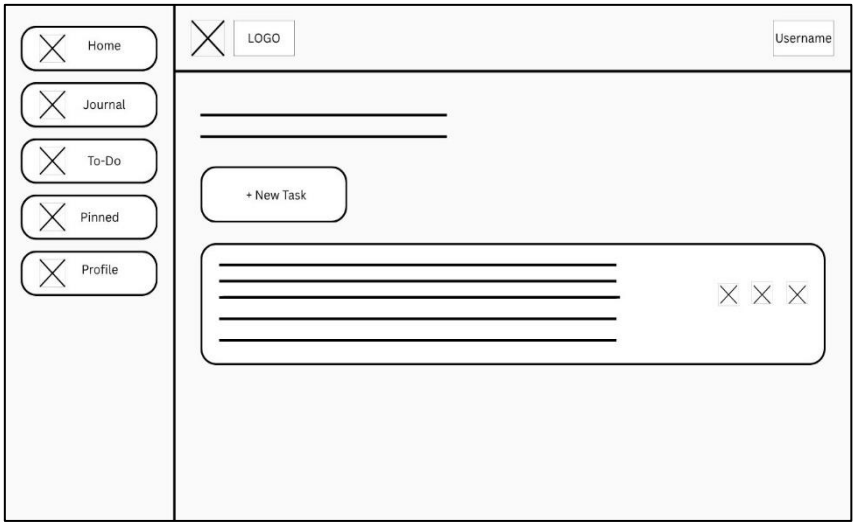

Figure 4.2.8: To-Do wireframe

5. System models

5.1 ER Diagram

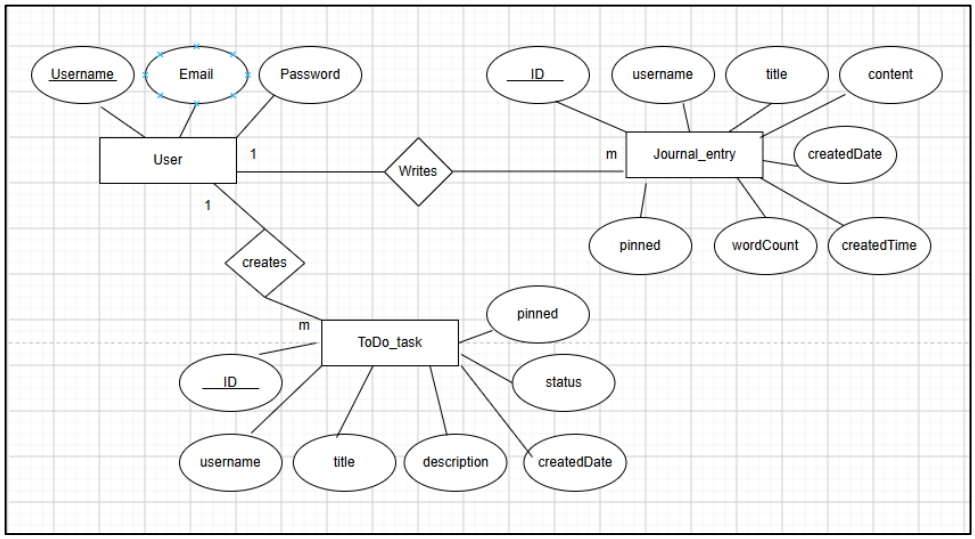

Figure 5.1.1: ERD

5.2 Functional Requirements

- The system shall allow a new user to register an account by providing a username, email, password, and password confirmation.
- The system shall redirect an already-logged-in user away from the Login/Register pages to the Dashboard.
- The system shall display a personalized welcome message containing the logged-in user's username.
- The system shall allow the user to create a new journal entry with a title and content.
- The system shall automatically calculate and store the word count of each journal entry's content.
- The system shall allow the user to create a new task with a title and an optional description.
- The system shall automatically record the creation date of each task.
- The system shall allow the user to pin or unpin a task.
- The system shall display all journal entries the user has pinned, in a dedicated section.
- The system shall display the logged-in user's current username and email.

- The system shall allow the user to permanently delete their account, after explicit confirmation.

5.3 Non-Functional Requirements

- **Usability -** The system shall present a clear, consistent visual design across all pages, using a unified color palette, typography, and component styling.

- **Reliability** - The system shall reliably persist all user data across browser sessions for as long as the browser's localStorage is not cleared.

- **Availability** - The system shall be available for use at any time without dependency on server uptime, as it runs entirely within the client's browser.

- **Security** - The system shall allow access to the Dashboard, Journal, To-Do, Pinned, and Profile pages to authenticated (logged-in) users only.

- **Maintainability** - The system's code shall follow the MVC architectural pattern, separating data logic (storage.js), presentation (HTML/CSS), and interaction logic (page-specific JavaScript files), to support future maintenance and extension.

- **Portability** - The system shall function correctly on any modern desktop without requiring installation.